# Lezione: Peter Corke – Robotics Toolbox (python)

## Fondamenti di Robotica


Docenti: Loredana Zollo
Clemente Lauretti
Tutor: Rita Molle








UNIVERSITA'
CAMPUS
BIO-MEDICO
DI ROMA


**CAMPUS BIO-MEDICO UNIVERSITY OF ROME**
Via Álvaro del Portillo, 21 - 00128 Rome - Italy
www.unicampus.it

# Outline

- Robotics Toolbox – Python
- Spatial math layer
- Robot models
- Cinematica diretta, inversa, pianificazione

# Robotics Toolbox – Python

- Libreria sviluppata per facilitare la simulazione e il controllo di robot nell'ambiente Python
- È parte del Progetto "**Robotics Toolbox**" creato da Peter Corke

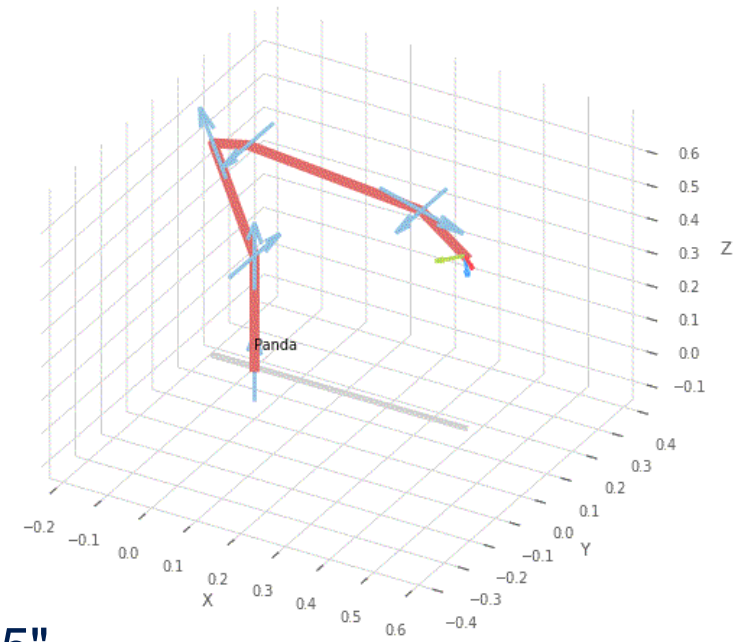
**Documentazione**:

- https://github.com/petercorke/robotics-toolbox-python/tree/master
- https://petercorke.github.io/robotics-toolbox-python/intro.html

**Tutorial**: https://github.com/jhavl/dkt

**Caratteristiche**

- Supporto per robot manipolatori
- Cinematica diretta e inversa
- Dinamica del robot
- Generazione di traiettorie
- Simulazione in 2D e 3D
- Supporto per ROS




```
python3 -m pip install --user --upgrade "numpy<1.25"
```

# Spatial math layer

La Spatial Math Layer permette di gestire trasformazioni spaziali e rotazioni → descrivere il movimento e l'orientamento dei robot

**Gruppi matematici:**

1. **SO(n) – Special Orthogonal group** → rotazioni nello spazio di dimensione n

*Rotazione di 90° attorno ad x*

```
from spatialmath import SO3
R = SO3.Rx(90, unit='deg')
print(R)
```

→

```
1    0    0
0    0   -1
0    1    0
```

2. **SE(n) – Special Euclidean group** → posizione + rotazione nello spazio di dimensione n

*Traslazione (1,2,3) +*
*Rotazione di 90° attorno a z*

```
from spatialmath import SE3
T = SE3(1, 2, 3) * SE3.Rz(90, unit='deg')
print(T)
```

→

```
0   -1    0    1
1    0    0    2
0    0    1    3
0    0    0    1
```

## Modi per rappresentare le rotazioni:

*Angoli di Eulero*

```
SO3.Eul([45, 30, 60], order='xyz', unit='deg')
```

*Quaternioni*

```
from spatialmath import UnitQuaternion
q = UnitQuaternion.RPY([45, 30, 60], unit='deg')
```

*Matrici di rotazione*

```
R = SO3.Ry(45, unit='deg')
```

*Roll-Pitch-Yaw*

```
SO3.RPY([45, 30, 60], order='zyx', unit='deg')
```

# Robot models

Nel toolbox ci sono 30 modelli di robot disponibili

I modelli cinematici possono essere specificati con:

- Denavit-Hartenberg
- Stringa Exponential Twist Representation (ETS)
- Albero rigido
- **File URDF**

```
import roboticstoolbox as rtb

robot = rtb.models.URDF.Panda()
print(robot)
```

1. Convertire xacro in urdf

```
xacro file.xacro > file.urdf
```

```
ERobot: panda (by Franka Emika), 7 joints (RRRRRRR), 1 gripper, geometry, collision
┌──────┬──────────────┬──────┬──────────────┬──────────────────────────────────────────────────────────────┐
│link  │     link     │joint │    parent    │                    ETS: parent to link                       │
├──────┼──────────────┼──────┼──────────────┼──────────────────────────────────────────────────────────────┤
│    0 │ panda_link0  │      │ BASE         │ SE3()                                                        │
│    1 │ panda_link1  │    0 │ panda_link0  │ SE3(0, 0, 0.333) ⊕ Rz(q0)                                    │
│    2 │ panda_link2  │    1 │ panda_link1  │ SE3(-90°, -0°, 0°) ⊕ Rz(q1)                                  │
│    3 │ panda_link3  │    2 │ panda_link2  │ SE3(0, -0.316, 0; 90°, -0°, 0°) ⊕ Rz(q2)                     │
│    4 │ panda_link4  │    3 │ panda_link3  │ SE3(0.0825, 0, 0; 90°, -0°, 0°) ⊕ Rz(q3)                     │
│    5 │ panda_link5  │    4 │ panda_link4  │ SE3(-0.0825, 0.384, 0; -90°, -0°, 0°) ⊕ Rz(q4)               │
│    6 │ panda_link6  │    5 │ panda_link5  │ SE3(90°, -0°, 0°) ⊕ Rz(q5)                                   │
│    7 │ panda_link7  │    6 │ panda_link6  │ SE3(0.088, 0, 0; 90°, -0°, 0°) ⊕ Rz(q6)                      │
│    8 │ @panda_link8 │      │ panda_link7  │ SE3(0, 0, 0.107)                                             │
└──────┴──────────────┴──────┴──────────────┴──────────────────────────────────────────────────────────────┘

┌──────┬──────┬──────┬──────┬──────┬──────┬──────┬──────┐
│name  │ q0   │ q1   │ q2   │ q3   │ q4   │ q5   │ q6   │
├──────┼──────┼──────┼──────┼──────┼──────┼──────┼──────┤
│  qr  │  0°  │ -17.2°│  0°  │ -126°│  0°  │ 115° │ 45°  │
│  qz  │  0°  │  0°  │  0°  │  0°  │  0°  │  0°  │  0°  │
└──────┴──────┴──────┴──────┴──────┴──────┴──────┴──────┘
```

# Cinematica diretta

Trovare la posizione e l'orientamento della terna utensile rispetto alla terna base, in funzione delle variabili di giunto $q$

Metodo della classe:

robot.fkine(*q, end=None, start=None, tool=None, include_base=True*)

**Input**:

q → variabili di giunto

end → end-effector rispetto al quale effettuare la cinematica diretta (se più di uno)

start → link di riferimento per la cinematica diretta

tool → trasformazione del tool

**Output**:

T → Matrice di trasformazione rappresentativa della posa dell'end-effector (tipo SE3)

Esempio:

```
Te = robot.fkine(robot.qz, end= 'panda_hand')
Te = robot.fkine([0, -0.3, 0, -2.2, 0, 2, 0.7854])
```

# Jacobiano geometrico

Matrice che mappa la velocità del giunto con la velocità spaziale dell'end-effector rispetto al base frame

Metodo della classe:

robot.jacob0(*q, end=None, start=None, tool=None*)

**Input**:

q → variabili di giunto

end → link di cui la velocità è descritta dallo Jacobiano

start → link di riferimento → base frame

tool → matrice di trasformazione statica del tool

**Output**:

J → Jacobiano del manipolatore nel frame «start» (tipo J0)

Esempio: J = robot.jacob0 ([0, 0, 0, 0, 0, 0, 0])

# Jacobiano analitico

Matrice che mappa la velocità del giunto con la velocità spaziale dell'end-effector rispetto al base frame

Metodo della classe:

robot.jacob0_analytical(*q, representation= 'rpy/xyz', end=None, start=None, tool=None*)

**Input**:

q → variabili di giunto

representation → rappresentazione angolare

end → link di cui la velocità è descritta dallo Jacobiano (default = base)

start → link di riferimento → end-effector

tool → matrice di trasformazione statica del tool

**Output**:

J → Jacobiano del manipolatore nel frame «start» (tipo J0)

Esempio: J = robot.jacob0_analytical ([0, 0, 0, 0, 0, 0, 0])

# Pianificatore di traiettoria

Generare una traiettoria polinomiale scalare

```
from roboticstoolbox import quintic
quintic(q0, qf, t, qd0=0, qdf=0)
```

**Input**:
q0 → valore iniziale (float)
qf → valore finale (float)
t → intero (n step) o vettore tempo
qd0 → velocità iniziale
qdf → velocità finale

**Output**:
tg → traiettoria

Esempio: `tg = quintic(q0,q1,m)`
→ traiettoria scalare mx1 che varia da q0 a qf

# Pianificatore di traiettoria

Generare una traiettoria polinomiale scalare come funzione

```
from roboticstoolbox import quintic_func
quintic_func(q0, qf, T, qd0=0, qdf=0)
```

**Input**:
q0 → valore iniziale (float)
qf → valore finale (float)
T → tempo di traiettoria (float)
qd0 → velocità iniziale
qdf → velocità finale

**Output**:
Funzione che calcola la «quintic polynomial», le sue derivate

Esempio: `f = quintic_func(q0,q1,T)`
→ Vedere f(0), f(5), f(2.5)… f(x)

# Pianificatore di traiettoria

Generare una traiettoria trapezoidale scalare

from roboticstoolbox import trapezoidal

trapezoidal(*q0, qf, t, V*)

**Input**:

q0 → valore iniziale (float)

qf → valore finale (float)

t → float (durata) o vettore tempo

V → velocità di segmenti lineari

**Output**:

tg → traiettoria

Esempio: tg = trapezoidal(1,2,10)

→ tg ha 10 time steps tra 1 e 2

# Pianificatore di traiettoria

Generare una traiettoria trapezoidale scalare come funzione

```
from roboticstoolbox import trapezoidal_func
trapezoidal_func(q0, qf, T, V)
```

**Input**:
q0 → valore iniziale (float)
qf → valore finale (float)
T → tempo massimo (float)
V → velocità di segmenti lineari

**Output**:
Profilo di funzione trapezoidale

Esempio:

```
f = trapezoidal_func(1,2,5)
```

→ Vedere f(0), f(5), f(2.5)… f(x)

# Pianificatore di traiettoria

Traiettoria multi-asse:

`robot.mtraj(tfunc, q0, qf, t)`

**Input**:

tfunc → traiettoria 1D (funzione, eg. quintic(), trapezoidal())
q0 → configurazione iniziale (array(m))
qf → configurazione finale (array(m))
t → vettore tempo o numero di step

**Output**:

tg → traiettoria multiasse

# Pianificatore di traiettoria

Traiettoria nello spazio dei giunti:
robot.jtraj(*q0, qf, t, qd0, qd1*)

**Input**:
q0 → variabili di giunto iniziali
qf → variabili di giunto finali
t → vettore tempo o numero di step
qd0 → velocità iniziale
qd1 → velocità finale

**Output**:
traj → oggetto traiettoria il cui attributo traj.q è la traiettoria nello spazio dei giunti

Esempio: traj = robot.jtraj (q0, qf, 50)

# Pianificatore di traiettoria

Traiettoria nello spazio dei giunti

robot.jtraj(*T1, T2, t, **kwargs*)

**Input**:

T1 → posa end-effector iniziale o variabili di giunto iniziali

T2 → posa end-effector finale o variabili di giunto finali

t → vettore tempo o numero di step

kwargs → argomenti passati per la cinematica inversa

tool → matrice di trasformazione statica del tool

**Output**:

traj → oggetto traiettoria il cui attributo traj.q è la traiettoria nello spazio dei giunti

Esempio: traj = robot.jtraj (T1, T2, 50)

# Pianificatore di traiettoria

Traiettoria cartesiana tra 2 pose

robot.ctraj(*T1, T2, t, s*)

**Input**:

T1 → posa end-effector iniziale (SE3)

T2 → posa end-effector finale (SE3)

t → vettore tempo (n) o numero di step

s → array di distanze lungo il path, nell'intervallo [0,1]

**Output**:

T0 → path (SE3 con n valori) da T1 a T2

Esempio: tg = ctraj(SE3.Rand(), SE3.Rand(), 20)

# Pianificatore di traiettoria

Traiettoria multi-segmento multi-asse

robot.mstraj(*viapoints, dt, tacc, qdmax=None, tsegment=None, q0=None, qd0=None, qdf=None, verbose=False*)

**Input**:
viapoints → set di viapoints, uno per riga
dt → time step
tacc → tempo di accelerazione
qdmax → velocità massima
tsegment → tempo massimo di ogni segmento
q0 → coordinate iniziali
qd0 → velocità iniziale; qdf → velocità finale
verbose → stampa informazioni di debug

**Output**:
traiettoria

# Inversione cinematica

Inversione cinematica numerica (output nello spazio dei giunti)

robot.IK.IKSolver(name='IK Solver', ilimit=30, slimit=100, tol=1e-06, mask=None, joint_limits=True, seed=None)

**Input**:

name → nome dell'algoritmo IK

ilimit → iterazioni permesse prima che inizi una nuova ricerca

slimit → ricerche permesse prima di dichiarare insuccesso

tol → errore residuo massimo

mask → 6 vector che assegna pesi all'errore di priorità dei gradi di libertà cartesiani

joint_limits → rigetta soluziomi che violano i limiti di giunto

seed → genera vettori casuali di coordinate di giunto

Methods

| | |
|---|---|
| `step` (ets, Tep, q) | Abstract step method |
| `solve` (ets, Tep[, q0]) | Solves the IK problem |
| `error` (Te, Tep) | Calculates the error between Te and Tep |

# Inversione – Levemberg-Marquadt Numerical IK (1)

Inversione cinematica numerica usando il metodo Levemberg-Marquadt (output nello spazio dei giunti)

robot.IK.IK_LM(name='IK Solver', ilimit=30, slimit=100, tol=1e-06, mask=None, joint_limits=True, seed=None, k=1.0, method='chan', kq=0.0, km=0.0, ps=0.0, pi=0.3, **kwargs)

**Input**:

name → nome dell'algoritmo IK

ilimit → iterazioni permesse prima che inizi una nuova ricerca

slimit → ricerche permesse prima di dichiarare insuccesso

tol → errore residuo massimo

mask → 6 vector che assegna pesi all'errore di priorità dei gradi di libertà cartesiani

joint_limits → rigetta soluziomi che violano i limiti di giunto

seed → genera vettori casuali di coordinate di giunto

k → valore di guadagno

method → «chan», «sugihara», «wampler» → definisce il metodo usato per calcolare la matrice di smorzamento

kq → guadagno per evitare i limiti di giunto; km → guadagno per massimizzazione

ps → angolo/distanza minima in cui il giunto può avvicinarsi al suo limite

pi → angolo/distanza di influenza in cui il movimento spaziale nullo diventa attivo

# Inversione – Levemberg-Marquadt Numerical IK (1)

## Esempio:

```
import roboticstoolbox as rtb
panda = rtb.models.Panda().ets()
solver = rtb.IK_LM()
Tep = panda.fkine([0, -0.3, 0, -2.2, 0, 2, 0.7854])
solver.solve(panda, Tep)
```

*IKSolution(q=array([-1.1097, -0.5518,  0.9276, -2.176 ,  0.483 ,  1.8918,  0.4071]), success=True, iterations=13, searches=1, residual=1.722003745997042e-10, reason='Success')*

Methods

| | |
|---|---|
| `step` (ets, Tep, q) | Abstract step method |
| `solve` (ets, Tep[, q0]) | Solves the IK problem |
| `error` (Te, Tep) | Calculates the error between Te and Tep |

# Inversione – Levemberg-Marquadt Numerical IK (2)

Inversione cinematica numerica → metodo Levemberg-Marquadt (output nello spazio dei giunti)

robot.ikine_LM(Tep, end=None, start=None, q0=None, ilimit=30, slimit=100, tol=1e-06, mask=None, joint_limits=True, seed=None, k=1.0, method='chan', kq=0.0, km=0.0, ps=0.0, pi=0.3, **kwargs)

**Input**:

Tep → posa end-effector desiderata SE3; end → link end-effector; start → link base

q0 → vettore coordinate giunti iniziale

ilimit → iterazioni permesse prima che inizi una nuova ricerca

slimit → ricerche permesse prima di dichiarare insuccesso

tol → errore residuo massimo

mask → 6 vector che assegna pesi all'errore di priorità dei gradi di libertà cartesiani

joint_limits → rigetta soluziomi che violano i limiti di giunto

seed → genera vettori casuali di coordinate di giunto

k → valore di guadagno

method → «chan», «sugihara», «wampler» → definisce il metodo usato per calcolare la matrice di smorzamento

kq → guadagno per evitare i limiti di giunto; km → guadagno per massimizzazione

ps → angolo/distanza minima in cui il giunto può avvicinarsi al suo limite

pi → angolo/distanza di influenza in cui il movimento spaziale nullo diventa attivo

# Inversione – Levemberg-Marquadt Numerical IK (2)

## Esempio:

```
import roboticstoolbox as rtb
panda = rtb.models.Panda()
Tep = panda.fkine([0, -0.3, 0, -2.2, 0, 2, 0.7854])
panda.ikine_LM(Tep)
```

*IKSolution(q=array([ 0.3059, -0.3126, -0.2739, -2.199 , -0.0959,  1.9959,  0.8611]), success=True, iterations=8, searches=1, residual=4.9981287487914 8e-07, reason='Success')*

# Inversione – Quadratic Programming Numerical IK (1)

Inversione cinematica numerica usando il metodo Quadratic Programming (output nello spazio dei giunti)

robot.IK.IK_QP(name='IK Solver', ilimit=30, slimit=100, tol=1e-06, mask=None, joint_limits=True, seed=None, kj=0.01, ks=1.0, kq=0.0, km=0.0, ps=0.0, pi=0.3, **kwargs)

**Input**:

name → nome dell'algoritmo IK

ilimit → iterazioni permesse prima che inizi una nuova ricerca

slimit → ricerche permesse prima di dichiarare insuccesso

tol → errore residuo massimo

mask → 6 vector che assegna pesi all'errore di priorità dei gradi di libertà cartesiani

joint_limits → rigetta soluziomi che violano i limiti di giunto

seed → genera vettori casuali di coordinate di giunto

kj → valore di guadagno per minimizzare la norma della velocità dei giunti

ks → guadagno che «aggiusta» l'errore intenzionale

kq → guadagno per evitare i limiti di giunto; km → guadagno per massimizzazione

ps → angolo/distanza minima in cui il giunto può avvicinarsi al suo limite

pi → angolo/distanza di influenza in cui il movimento spaziale nullo diventa attivo

# Inversione – Quadratic Programming Numerical IK (1)

## Methods

| | |
|---|---|
| `step` (ets, Tep, q) | Abstract step method |
| `solve` (ets, Tep[, q0]) | Solves the IK problem |
| `error` (Te, Tep) | Calculates the error between Te and Tep |

# Inversione – Quadratic Programming Numerical IK (2)

Inversione cinematica numerica usando il metodo Quadratic Programming (output nello spazio dei giunti)

robot.IK.IK_QP(Tep, end=None, start=None, q0=None, ilimit=30, slimit=100, tol=1e-06, mask=None, joint_limits=True, seed=None, kj=0.01, ks=1.0, kq=0.0, km=0.0, ps=0.0, pi=0.3, **kwargs)

**Input**:

Tep → posa end-effector desiderata SE3; end → link end-effector; start → link base
q0 → vettore coordinate giunti iniziale
ilimit → iterazioni permesse prima che inizi una nuova ricerca
slimit → ricerche permesse prima di dichiarare insuccesso
tol → errore residuo massimo
mask → 6 vector che assegna pesi all'errore di priorità dei gradi di libertà cartesiani
joint_limits → rigetta soluziomi che violano i limiti di giunto
seed → genera vettori casuali di coordinate di giunto
kj → valore di guadagno per minimizzare la norma della velocità dei giunti
ks → guadagno che «aggiusta» l'errore intenzionale
kq → guadagno per evitare i limiti di giunto; km → guadagno per massimizzazione
ps → angolo/distanza minima in cui il giunto può avvicinarsi al suo limite
pi → angolo/distanza di influenza in cui il movimento spaziale nullo diventa attivo

# Inversione – Gauss-Newton Numerical IK (1)

Inversione cinematica numerica usando il metodo Gauss-Newton (output nello spazio dei giunti)

robot.IK.IK_GN(name='IK Solver', ilimit=30, slimit=100, tol=1e-06, mask=None, joint_limits=True, seed=None, pinv=False, kq=0.0, km=0.0, ps=0.0, pi=0.3, **kwargs)

**Input**:

name → nome dell'algoritmo IK

ilimit → iterazioni permesse prima che inizi una nuova ricerca

slimit → ricerche permesse prima di dichiarare insuccesso

tol → errore residuo massimo

mask → 6 vector che assegna pesi all'errore di priorità dei gradi di libertà cartesiani

joint_limits → rigetta soluziomi che violano i limiti di giunto

seed → genera vettori casuali di coordinate di giunto

pinv → usa la pseudoinversa se settata a True

kq → guadagno per evitare i limiti di giunto; km → guadagno per massimizzazione

ps → angolo/distanza minima in cui il giunto può avvicinarsi al suo limite

pi → angolo/distanza di influenza in cui il movimento spaziale nullo diventa attivo

# Inversione – Gauss-Newton Numerical IK (1)

## Esempio:

```
import roboticstoolbox as rtb
panda = rtb.models.Panda().ets()
solver = rtb.IK_GN(pinv=True)
Tep = panda.fkine([0, -0.3, 0, -2.2, 0, 2, 0.7854])
solver.solve(panda, Tep)
```

*IKSolution(q=array([ 1.2886,  1.4862, -2.1934, -2.0771,  1.244 ,  1.1468, -0.096 ]), success=True, iterations=53, searches=4, residual=9.796561164911834e-10, reason='Success')*

Methods

| | |
|---|---|
| `step` (ets, Tep, q) | Abstract step method |
| `solve` (ets, Tep[, q0]) | Solves the IK problem |
| `error` (Te, Tep) | Calculates the error between Te and Tep |

# Inversione – Gauss-Newton Numerical IK (2)

Inversione cinematica numerica usando il metodo Gauss-Newton (output nello spazio dei giunti)

robot.IK.IK_GN(Tep, end=None, start=None, q0=None, ilimit=30, slimit=100, tol=1e-06, mask=None, joint_limits=True, seed=None, pinv=False, kq=0.0, km=0.0, ps=0.0, pi=0.3, **kwargs)

**Input**:

Tep → posa end-effector desiderata SE3; end → link end-effector; start → link base

q0 → vettore coordinate giunti iniziale

ilimit → iterazioni permesse prima che inizi una nuova ricerca

slimit → ricerche permesse prima di dichiarare insuccesso

tol → errore residuo massimo

mask → 6 vector che assegna pesi all'errore di priorità dei gradi di libertà cartesiani

joint_limits → rigetta soluziomi che violano i limiti di giunto

seed → genera vettori casuali di coordinate di giunto

pinv → usa la pseudoinversa se settata a True

kq → guadagno per evitare i limiti di giunto; km → guadagno per massimizzazione

ps → angolo/distanza minima in cui il giunto può avvicinarsi al suo limite

pi → angolo/distanza di influenza in cui il movimento spaziale nullo diventa attivo

# Inversione – Gauss-Newton Numerical IK (2)

## Esempio:

```
import roboticstoolbox as rtb
panda = rtb.models.Panda()
Tep = panda.fkine([0, -0.3, 0, -2.2, 0, 2, 0.7854])
panda.ikine_GN(Tep)
```

*IKSolution(q=array([ 2.2611, -1.4514, -2.8337, -0.6221, -2.6177,  1.1555, -2.4071]), success=False, iterations=100, searches=100, residual=0.0, reason='iteration and search limit reached, 100 numpy.LinAlgError encountered')*

# Inversione – Newton-Raphson Numerical IK (1)

Inversione cinematica numerica usando il metodo Newton-Raphson (output nello spazio dei giunti)

robot.IK.IK_NR(name='IK Solver', ilimit=30, slimit=100, tol=1e-06, mask=None, joint_limits=True, seed=None, pinv=False, kq=0.0, km=0.0, ps=0.0, pi=0.3, **kwargs)

**Input**:

name → nome dell'algoritmo IK

ilimit → iterazioni permesse prima che inizi una nuova ricerca

slimit → ricerche permesse prima di dichiarare insuccesso

tol → errore residuo massimo

mask → 6 vector che assegna pesi all'errore di priorità dei gradi di libertà cartesiani

joint_limits → rigetta soluziomi che violano i limiti di giunto

seed → genera vettori casuali di coordinate di giunto

pinv → usa la pseudoinversa se settata a True → deve essere true se >6DoF

kq → guadagno per evitare i limiti di giunto; km → guadagno per massimizzazione

ps → angolo/distanza minima in cui il giunto può avvicinarsi al suo limite

pi → angolo/distanza di influenza in cui il movimento spaziale nullo diventa attivo

# Inversione – Newton-Raphson Numerical IK (1)

## Esempio:

```
import roboticstoolbox as rtb
panda = rtb.models.Panda().ets()
solver = rtb.IK_NR(pinv=True)
Tep = panda.fkine([0, -0.3, 0, -2.2, 0, 2, 0.7854])
solver.solve(panda, Tep)
```

*IKSolution(q=array([-2.3853,  0.3909,  2.4784, -2.1923, -0.27 ,  1.9673,  0.9981]), success=True, iterations=188, searches=12, residual=2.4219174319660 1e-10, reason='Success')*

Methods

| | |
|---|---|
| `step` (ets, Tep, q) | Abstract step method |
| `solve` (ets, Tep[, q0]) | Solves the IK problem |
| `error` (Te, Tep) | Calculates the error between Te and Tep |

# Inversione – Newton-Raphson Numerical IK (2)

Inversione cinematica numerica usando il metodo Newton-Raphson (output nello spazio dei giunti)

robot.IK.IK_NR(Tep, end=None, start=None, q0=None, ilimit=30, slimit=100, tol=1e-06, mask=None, joint_limits=True, seed=None, pinv=False, kq=0.0, km=0.0, ps=0.0, pi=0.3, **kwargs)

**Input**:

Tep → posa end-effector desiderata SE3; end → link end-effector; start → link base
q0 → vettore coordinate giunti iniziale
ilimit → iterazioni permesse prima che inizi una nuova ricerca
slimit → ricerche permesse prima di dichiarare insuccesso
tol → errore residuo massimo
mask → 6 vector che assegna pesi all'errore di priorità dei gradi di libertà cartesiani
joint_limits → rigetta soluziomi che violano i limiti di giunto
seed → genera vettori casuali di coordinate di giunto
pinv → usa la pseudoinversa se settata a True → deve essere true se >6DoF
kq → guadagno per evitare i limiti di giunto; km → guadagno per massimizzazione
ps → angolo/distanza minima in cui il giunto può avvicinarsi al suo limite
pi → angolo/distanza di influenza in cui il movimento spaziale nullo diventa attivo

# Inversione – Newton-Raphson Numerical IK (2)

## Esempio:

```
import roboticstoolbox as rtb
panda = rtb.models.Panda()
Tep = panda.fkine([0, -0.3, 0, -2.2, 0, 2, 0.7854])
panda.ikine_NR(Tep)
```

*IKSolution(q=array([-2.5448,  0.0635, -0.6093, -2.5018, -1.2853,  1.3894, -0.2102]), success=False, iterations=100, searches=100, residual=0.0, reason='iteration and search limit reached, 100 numpy.LinAlgError encountered')*

# Plot

Metodo della classe:

robot.plot(*q, backend=None, block=False, dt=0.05, limits=None, vellipse=False, fellipse=False, fig=None, movie=None, loop=False, **kwargs*)

**Input**:

q → variabili di giunto

backend → backend da usare → pyplot o swift

block → mantiene la figura aperta

dt → se q è una traiettoria, è il ritardo tra frames

limits → limiti del plot

vellipse, fellipse, fig, movie, loop, **kwargs → opzioni per il plot

**Output**:

env → un riferimento all'oggetto ambiente che controlla la figure (tipo env)

Esempio: robot.plot (q, 'pyplot')